



VMES Grinds





**30. Substring with concatenation of
all words**



30. Substring with Concatenation of All Words

Solved

Hard

Topics

Companies

You are given a string `s` and an array of strings `words`. All the strings of `words` are of **the same length**.

A **concatenated string** is a string that exactly contains all the strings of any permutation of `words` concatenated.

- For example, if `words = ["ab","cd","ef"]`, then `"abcdef"`, `"abefcd"`, `"cdabef"`, `"cdefab"`, `"efabcd"`, and `"efcdab"` are all concatenated strings. `"acdbef"` is not a concatenated string because it is not the concatenation of any permutation of `words`.

Return an array of *the starting indices* of all the concatenated substrings in `s`. You can return the answer in **any order**.



Example 1:

Input: `s = "barfoothefoobarman"`, `words = ["foo","bar"]`

Output: `[0,9]`

Explanation:

The substring starting at 0 is `"barfoo"`. It is the concatenation of `["bar","foo"]` which is a permutation of `words`.

The substring starting at 9 is `"foobar"`. It is the concatenation of `["foo","bar"]` which is a permutation of `words`.

- Position 0: `"barfoo"` = `"bar"` + `"foo"`
- Position 9: `"foobar"` = `"foo"` + `"bar"`



Example 2:

Input: `s = "wordgoodgoodgoodbestword", words = ["word","good","best","word"]`

Output: `[]`

Explanation:

There is no concatenated substring.

- Can't find all 4 words together anywhere



Example 3:

Input: `s = "barfoofoobarthefoobarman", words = ["bar","foo","the"]`

Output: `[6,9,12]`

Explanation:

The substring starting at 6 is "foobarthe". It is the concatenation of ["foo","bar","the"].

The substring starting at 9 is "barthefoo". It is the concatenation of ["bar","the","foo"].

The substring starting at 12 is "thefoobar". It is the concatenation of ["the","foo","bar"].

- Position 6: "foobarthe"
- Position 9: "barthefoo"
- Position 12: "thefoobar"



Constraints:

- `1 <= s.length <= 104`
- `1 <= words.length <= 5000`
- `1 <= words[i].length <= 30`
- `s` and `words[i]` consist of lowercase English letters.



Explanation



Goal

- Find all starting indices in string “s”
- The substring must be a concatenation of all words in the list
- Word can appear in any order
- No extra characters allowed between words

- **Try position 0, 1, 2, 3... one by one**
- **At each position, grab all the words**
- **Count what you grabbed**
- **See if it matches what you need**
- **If yes, save that position**



Initial Approach



```
class Solution(object):
    def findSubstring(self, s, words):

        if not s or not words:
            return []

        word_len = len(words[0])
        total_len = word_len * len(words)

        target = {}
        for w in words:
            target[w] = target.get(w, 0) + 1

        result = []

        for i in range(len(s) - total_len + 1):
            seen = {}

            for j in range(0, total_len, word_len):
                word = s[i + j: i + j + word_len]
                seen[word] = seen.get(word, 0) + 1

            if seen == target:
                result.append(i)

        return result
```



Optimized Approach



Problem with initial solution

Checking the same thing over and over:

Position 0: check "bar" + "foo"

Position 1: check "arf" + "oot"

Position 2: check "rfo" + "oth"

Position 3: check "foo" + "the" ← We're rechecking "foo" again



Sliding windows

Train: [bar][foo][the][foo][bar]

Window 1: [bar][foo] ← Check this

Window 2: [foo][the] ← Just remove "bar", add "the"

Window 3: [the][foo] ← Just remove "foo", add "foo"

And Since all words are same length, only need to start from
3 different positions

Position 0: 0, 3, 6, 9, 12... (every 3rd position)

Position 1: 1, 4, 7, 10, 13... (every 3rd position)

Position 2: 2, 5, 8, 11, 14... (every 3rd position)



```
return result
```



Time and Space Complexity Analysis



```
class Solution(object):
    def findSubstring(self, s, words):

        if not s or not words:
            return []

        word_len = len(words[0])
        total_len = word_len * len(words)

        target = {}
        for w in words:
            target[w] = target.get(w, 0) + 1

        result = []

        for i in range(len(s) - total_len + 1):
            seen = {}

            for j in range(0, total_len, word_len):
                word = s[i + j: i + j + word_len]
                seen[word] = seen.get(word, 0) + 1

            if seen == target:
                result.append(i)

        return result
```

Space - $O(k)$ where k is number of unique words

Time - $O(n)$

Time - $O(m)$

Time - $O(n) \times O(m) = O(n \times m)$
where n is $\text{len}(s) - \text{total lens}$
but assume as " n ",
 m is word len



```
class Solution:
    def findSubstring(self, s, words):
        if not s or not words:
            return []
        word_len = len(words[0])
        # total_len = word_len * len(words)
        target = {}
        for w in words:
            target[w] = target.get(w, 0) + 1
        result = []
        for i in range(word_len):
            left = i
            right = i
            seen = {}
            count = 0
            while right + word_len <= len(s):
                w = s[right: right + word_len]
                right += word_len
                if w in target:
                    seen[w] = seen.get(w, 0) + 1
                    count += 1
                    while seen[w] > target[w]:
                        left_w = s[left: left + word_len]
                        left += word_len
                        seen[left_w] -= 1
                        count -= 1
                    if count == len(words):
                        result.append(left)
                else:
                    seen.clear()
                    count = 0
                    left = right
            return result
```

Time - $O(m)$

Time - $O(n/m)$

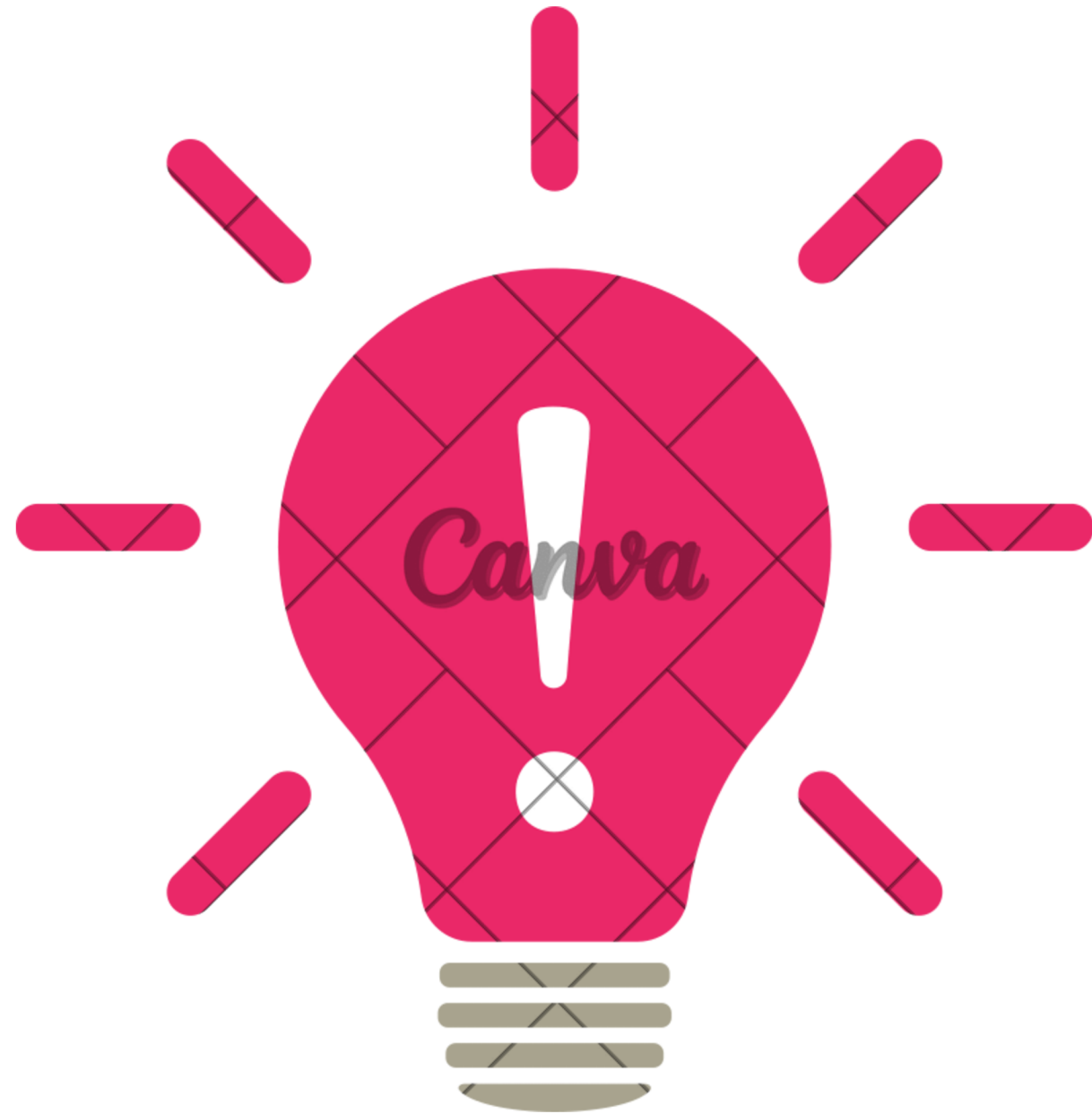
Space - $O(k)$ where k is
number of unique words

Time - $O(m) \times O(n/m) =$
 $O(n)$ <---
where n is $\text{len}(s)$ - total lens
but assume as " n ",
 m is word len



Key Takeaway

VMES Grinds MMM
Chee sar



- HashMaps are good for tracking frequency and checking multiset equality in constant time.
- When elements' width is fixed, avoid character-by-character loop.
- Sliding Window Pattern: For contiguous subarray problems, dynamic sliding windows (expand right, shrink left) are almost always $O(N)$.